



机器学习 基础

提纲

一、机器学习基本概念

二、线性回归

三、支持向量机

四、K-means算法

五、PCA降维

提纲

一、机器学习基本概念

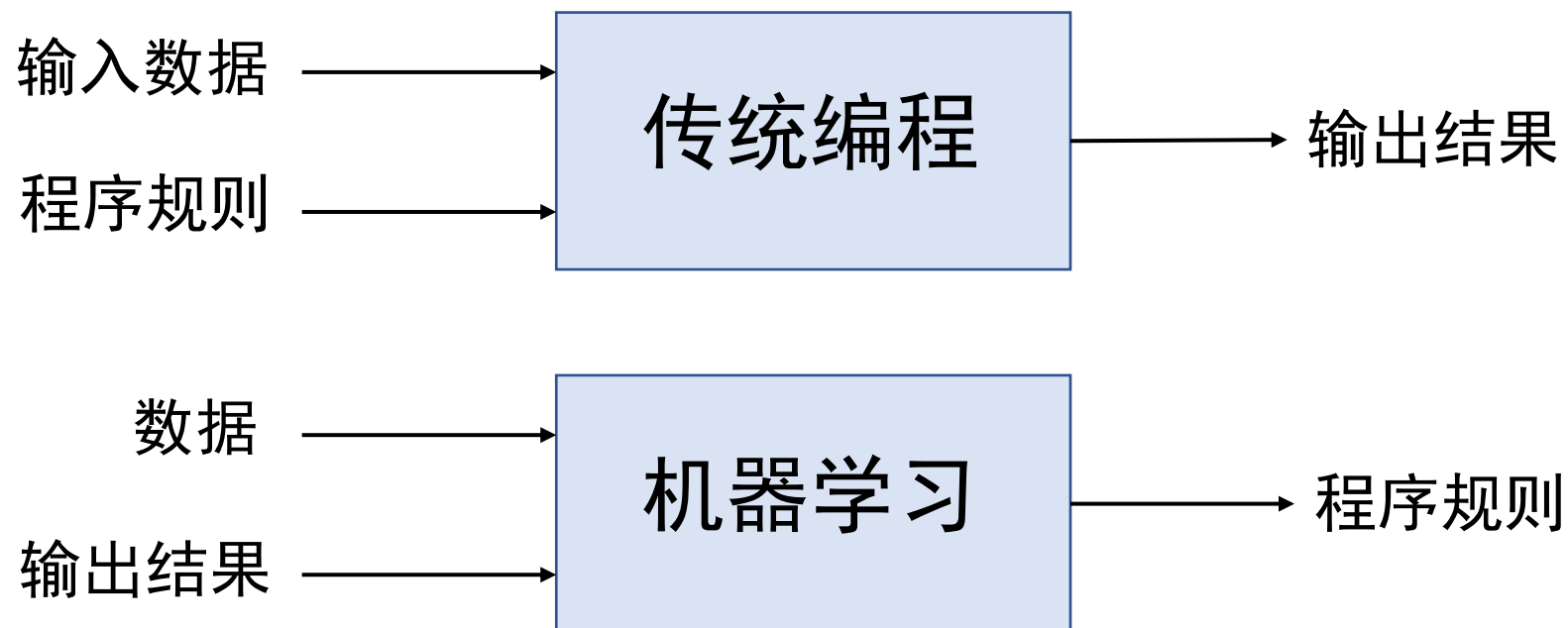
二、线性回归

三、支持向量机

四、K-means算法

五、PCA降维

机器学习研究和构建的是一类特殊算法，能够让计算机自己在数据中学习从而进行预测。



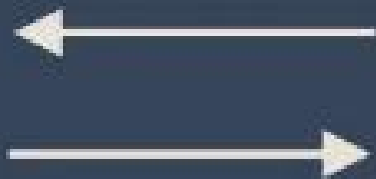
人工智能、机器学习、深度学习

人工智能

机器学习

深度学习

机器学习的基本思路



现实问题抽象为数学问题

机器解决数学问题

从而解决现实问题

机器学习的原理

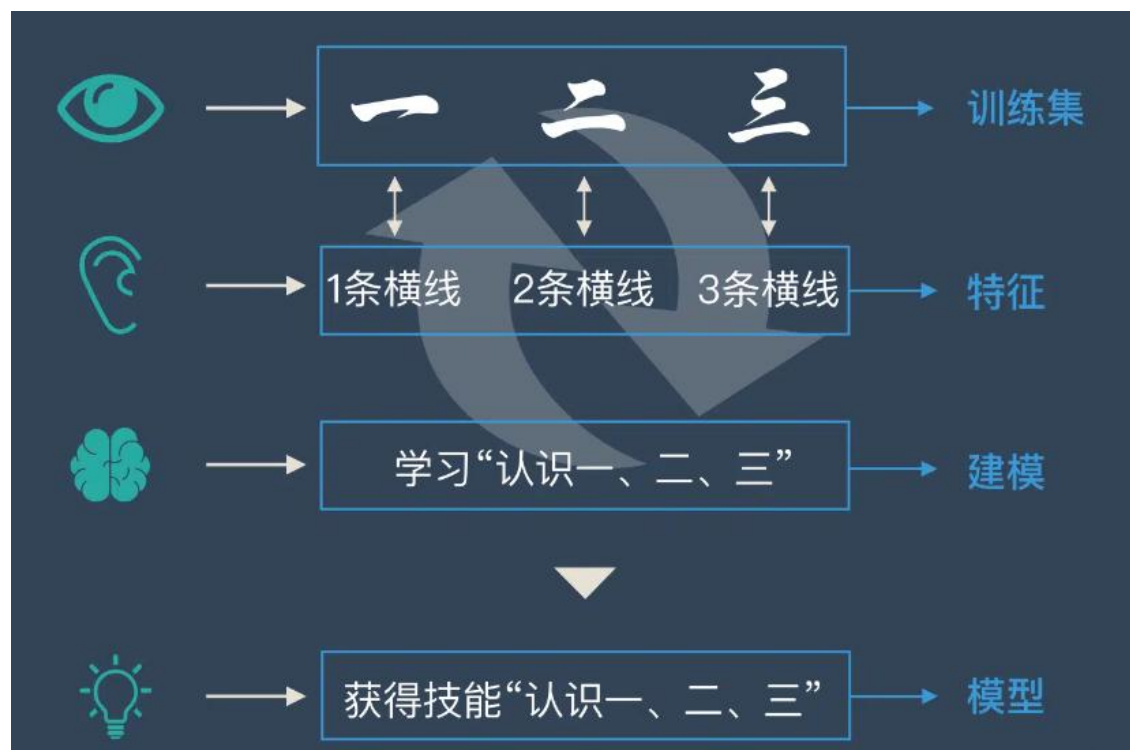
假如我们正在教小朋友识字（一、二、三）。我们首先会拿出3张卡片，然后便让小朋友看卡片，一边说“一条横线的是一、两条横线的是二、三条横线的是三”。

当重复的次数足够多时，小朋友就学会了一个新技能——认识汉字：一、二、三。



模型选择：小朋友可能用手指数数字的个数，机器学习会选择一个模型，这个模型就像一个**学习规则**，它可以帮助机器理解数据。

模型训练：小朋友会反复**数数字**，机器学习会用很多数据来**训练模型**，让它能够认识不同的特征。



预测和验证：在看过一定数量的数字后，小朋友可以看到一个新的数字，然后猜是不是一、二、三。机器学习也会用训练好的模型来预测新的数据，并验证它的猜测是否正确。



调整和优化：小朋友可能会发现自己猜错了，然后会调整自己的方法，机器学习也会根据预测结果来调整模型，让它变得更准确。



数字卡片集合： 假设小朋友有很多数字卡片， 每张卡片上有不同的数字。
我们把这些卡片分成三堆。



训练集 (Training Set)

这是第一堆卡片，就像小朋友反复数数字的练习一样。小朋友会用这些卡片学习认识数字，就像机器学习会用训练集来训练模型，让它学会认识数据中的特征。

验证集 (Validation Set)

这是第二堆卡片，就像小朋友在自己数数字时，用一些新的卡片来验证自己的方法一样。小朋友会用这些卡片来验证自己的猜测是否准确，就像机器学习会用验证集来验证模型的预测是否准确。

测试集 (Test Set)

这是第三堆卡片，就像小朋友在考试时，用一些全新的卡片来测试自己的能力一样。小朋友会用这些卡片来测试自己对数字的认识程度，就像机器学习会用测试集来测试模型的泛化能力，也就是看它在全新的数据上能否做出准确的预测。

机器学习的评估指标

准确率 (Accuracy)

在所有样本中，预测正确的概率

$$accuracy = \frac{TN + TP}{TN + TP + FN + FP}$$

$$precision = \frac{TP}{TP + FP}$$

精确率 (Precision)

你认为是正样本中，有多少是真的正确的概率

$$recall = \frac{TP}{TP + FN}$$

召回率 (Recall)

正样本中有多少是被找了出来

	预测值 0	预测值 1
真实值 0	TN	FP
真实值 1	FN	TP

机器学习的评估指标

准确率 (Accuracy)

在所有样本中，预测正确的概率

适用于数据类别相对平衡的情况。当各个类别的样本数量相近时，准确率可以给出整体分类的正确性。

精确率 (Precision)

你认为是正样本中，有多少是真的正确的概率

适用于关注分类为正例的准确性，特别是当假阳性的代价较高时。例如，当预测癌症时，你希望确保所预测的阳性样本是确实患有癌症的。

召回率 (Recall)

正样本中有多少被找了出来

适用于关注实际为正例的样本被正确地分类为正例的情况。在医疗诊断中，召回率更重要，因为你不希望错过任何一个实际阳性的样本。

	预测值 0	预测值 1
真实值 0	TN	FP
真实值 1	FN	TP

$$accuracy = \frac{TN + TP}{TN + TP + FN + FP}$$

$$precision = \frac{TP}{TP + FP}$$

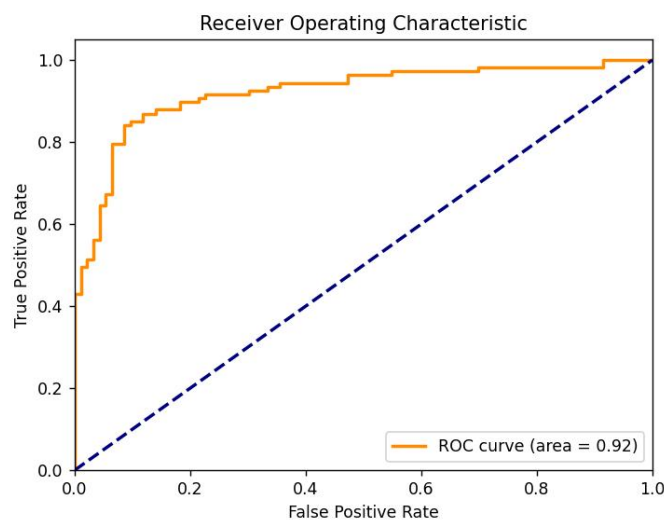
$$recall = \frac{TP}{TP + FN}$$

【例】训练一个癌细胞诊断模型，在训练数据中其中负样本（非癌细胞）有10万个，而正样本（癌细胞）只有200个。假如某个模型将其中的105个预测为正样本，100095个预测为负样本。最终经过核对后发现，正样本中有5个预测正确，负样本中有99900个样本预测正确。

机器学习的评估指标

ROC曲线 (Receiver Operating Characteristic Curve)

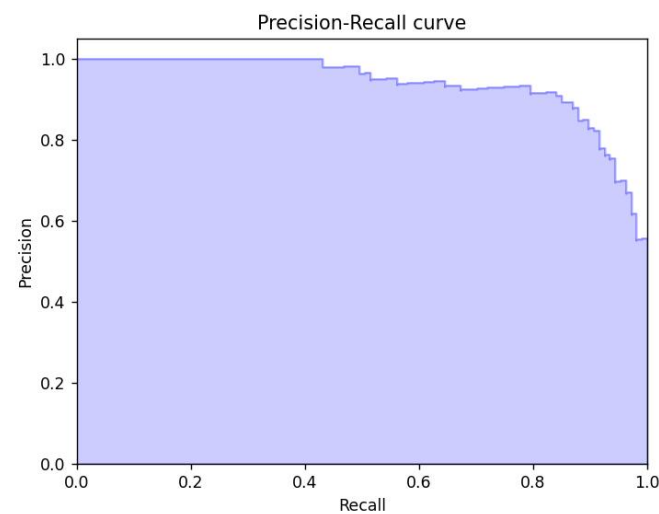
展示真阳性率与假阳性率之间的权衡关系，用于选择不同阈值下的性能。



	预测值 0	预测值 1
真实值 0	TN	FP
真实值 1	FN	TP

P-R曲线 (Precision-Recall Curve)

展示精确率与召回率之间的权衡关系，用于处理类别不平衡问题。



Example:评估指标的计算实现

【例】disp_roc_pr.py

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, auc, precision_recall_curve,
average_precision_score
import matplotlib.pyplot as plt
```

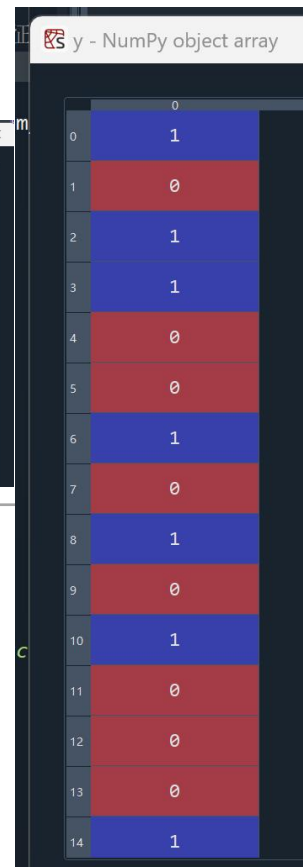
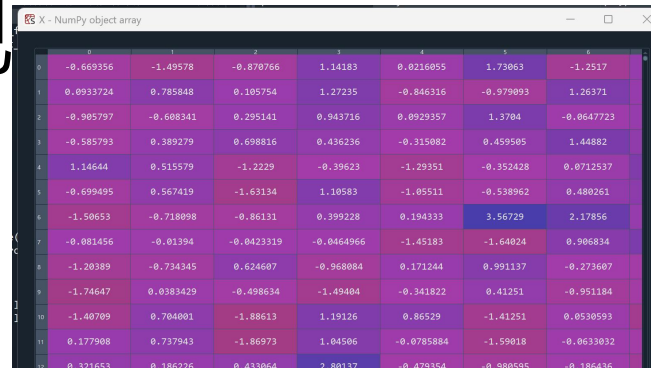
创建示例数据,创建具有两个或多个类别的数据集, 并控制各种参数, 例如样本数1000、20个特征

用随机种子控制数据集的划分

```
X, y = make_classification(n_samples=1000, n_features=20,
random_state=42)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

随机数生成器的种子,
确保每次分割数据时
结果相同。



训练模型 (这里使用逻辑回归作为示例)

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

预测概率

```
y_prob = model.predict_proba(X_test)[: , 1]
```

计算ROC曲线和AUC

```
fpr, tpr, _ = roc_curve(y_test, y_prob)
```

```
roc_auc = auc(fpr, tpr)
```

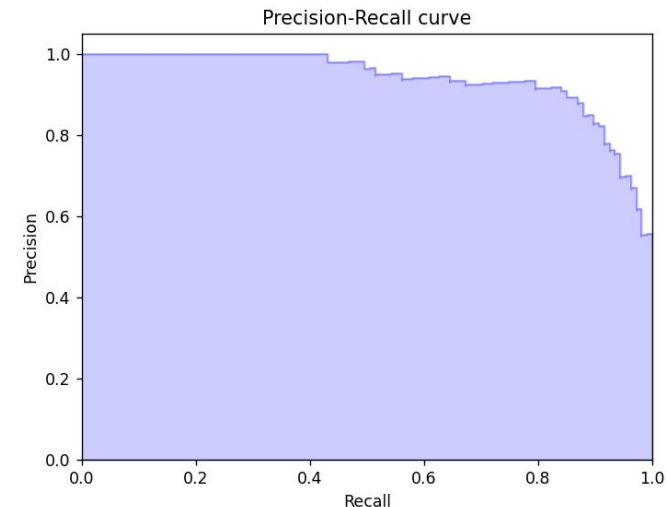
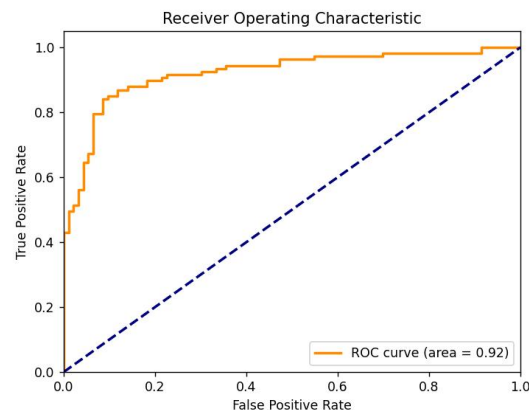
计算P-R曲线和AUC-PR

```
precision, recall, _ = precision_recall_curve(y_test, y_prob)
```

```
pr_auc = average_precision_score(y_test, y_prob)
```


Example:评估指标的计算实现

```
# 绘制ROC曲线
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()
```



```
# 绘制P-R曲线
plt.figure()
plt.step(recall, precision, color='b', alpha=0.2, where='post')
plt.fill_between(recall, precision, step='post', alpha=0.2, color='b')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.ylim([0.0, 1.05])
```


机器学习的评估指标

评估指标	Scikit-Learn代码示例
准确率(Accuracy)	<pre>from sklearn.metrics import accuracy_score accuracy_score(y_true,y_pred)</pre>
精确率(Precision)	<pre>from sklearn.metrics import precision_score precision_score(y_true, y_pred)</pre>
召回率(Recall)	<pre>from sklearn,metrics import recall_score recall_score(y_true, y_pred)</pre>
F1(F1-Score)	<pre>from sklearn.metrics import f1_score f1_score(y_true, y_pred)</pre>
ROC	<pre>from sklearn.metrics import roc_curve fpr, tpr, thresholds = roc_curve(y_true, y_prob)</pre>
PR	<pre>from sklearn.metrics import precision_recall_curve precision, recall, thresholds = precision_recall_curve(y_true, y_prob)</pre>

机器学习的分类

监督学习(supervised learning)

数据有标签、一般为回归或分类等任务

无监督学习(un-supervised learning)

数据无标签、一般为聚类或若干降维任务

强化学习(reinforcement learning)

序列数据决策学习，一般为与从环境交互中学习

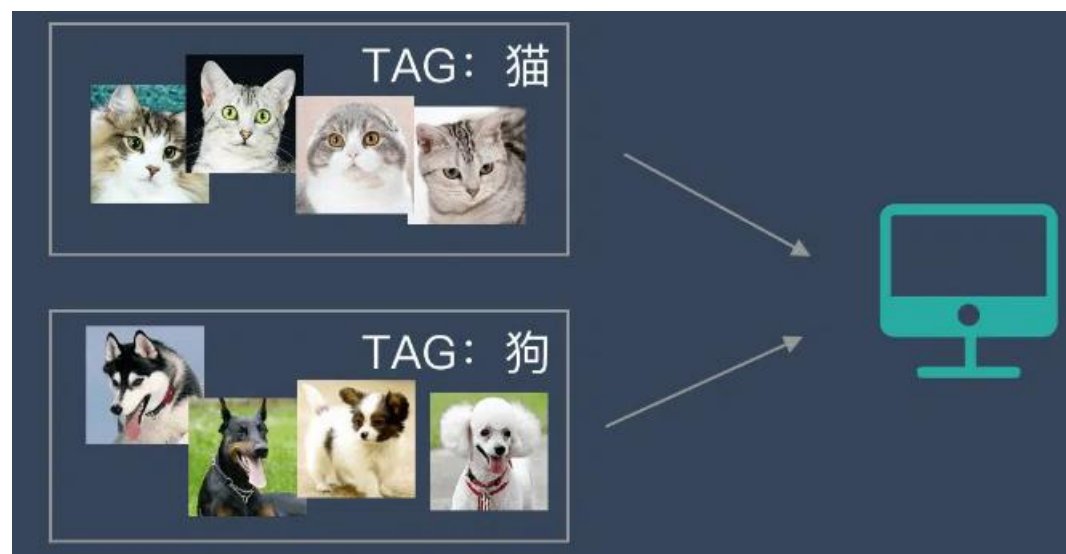
半监督学习

(semi-supervised learning)

监督学习

监督学习是指我们给算法一个数据集，并且给定正确答案。机器通过数据来学习正确答案的计算方法。

- 我们准备了一大堆猫和狗的照片，我们想让机器学会如何识别猫和狗。当我们使用监督学习的时候，我们需要给这些照片打上标签。
- 类比小朋友学习数字的过程，我们将训练集卡片中的数字教给小朋友，再让他们认识测试集上的数字以评估学习效果。



监督学习

监督学习的要素



```
graph LR; A[监督学习的要素] --- B[标注数据  
标识了类别信息的数据]; A --- C[学习模型  
如何学习得到优秀的模型]; A --- D[损失函数  
如何评估学习结果];
```

标注数据
标识了类别信息的数据

学习模型
如何学习得到优秀的模型

损失函数
如何评估学习结果

监督学习--损失函数

- 损失函数 $Loss$ 就是用来计算 x_i 真值 y_i 与预测值 $f(x_i)$ 之间差值的函数。
- 在训练过程中希望优化映射函数，使得在训练数据集上得到“损失”之和最小，即

$$\min \sum_{i=1}^n Loss(f(x_i), y_i)。$$

训练数据集
 $(x_i, y_i), i = 1, \dots, n$

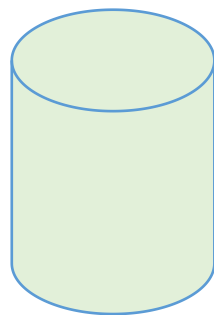
损失函数名称	损失函数定义
0-1损失函数	$Loss(y_i, f(x_i)) = \begin{cases} 1, & f(x_i) \neq y_i \\ 0, & f(x_i) = y_i \end{cases}$
平方损失函数	$Loss(y_i, f(x_i)) = (y_i - f(x_i))^2$
绝对损失函数	$Loss(y_i, f(x_i)) = y_i - f(x_i) $
对数损失函数/ 对数似然损失函数	$Loss(y_i, P(y_i x_i)) = -\log P((y_i x_i))$

典型的损失函数

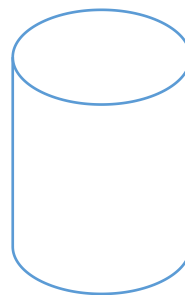
监督学习--训练集和测试集

从训练集学习得到
映射函数 f 的未知参数

在测试数据集
测试映射函数 f



训练数据集
 $(x_i, y_i), i = 1, \dots, n$



测试数据集
 $(x_i', y_i'), i = 1, \dots, m$

监督学习--常见算法

回归

定量输出，连续变量预测

线性回归 (Linear Regression)

线性回归就是要找一条直线，并且让这条直线尽可能地拟合所有数据点。

逻辑回归 (Logistic Regression)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

主要用于二分类问题， x 为自变量，函数值为取0或1的概率

分类

定性输出，离散变量预测

决策树 (Decision Trees)

递归地选择最优特征，并根据该特征对训练数据集进行分割，使得对各个子数据集有一个最好的分类过程，这一过程对应着对特征空间的划分，也对应着决策树的构建。

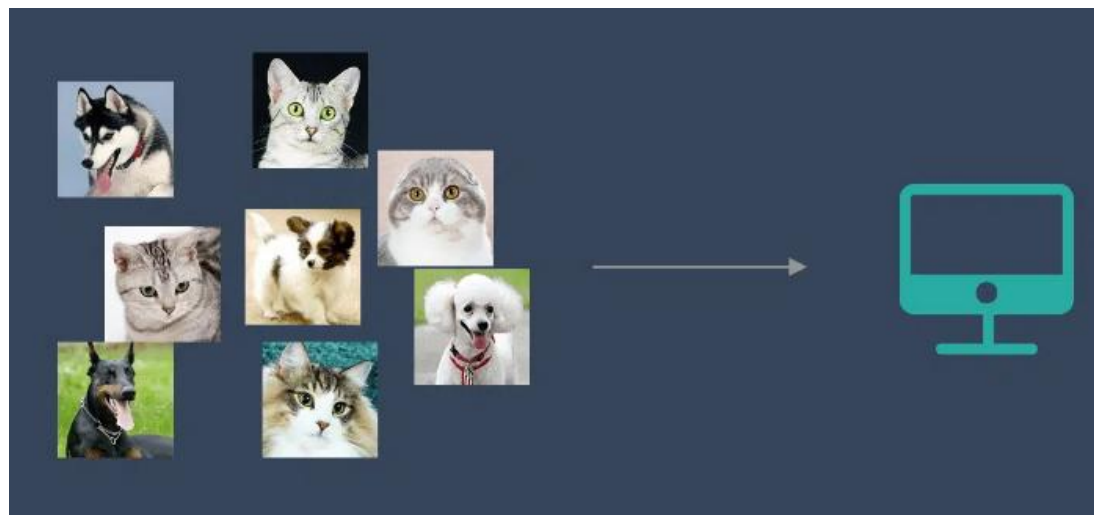
朴素贝叶斯 (Naive Bayes)

基于贝叶斯定理与特征条件独立假设的分类方法。对于给定的训练数据集，首先基于特征条件独立假设学习输入输出的联合概率分布。然后基于此模型，对给定的输入 x ，利用贝叶斯定理求出后验概率最大的输出 y 。

非监督学习

非监督学习中，给定的数据集没有“正确答案”，所有的数据都是一样的。无监督学习的任务是从给定的数据集中，挖掘出潜在的结构。

- 我们把一堆猫和狗的照片给机器，不给这些照片打任何标签，但是我们希望机器能够将这些照片分分类。
- 类比小朋友学习数字的过程，非监督学习则是将卡片一起扔给小朋友，让他们将长得像的卡片分为一类。



非监督学习--聚类

非监督学习的算法主要以聚类算法为代表，常见的聚类算法有：

- **K均值聚类 (K-Means Clustering)**

应用场景：适用于数据集中明显的球状或类似球状的分布，每个簇的大小相似，并且簇之间的分隔较为清晰。常用于市场分割、图像压缩和文本聚类等。

- **层次聚类 (Hierarchical Clustering)**

应用场景：适用于具有层次结构的数据，可以生成树状的聚类图。不需要事先指定簇的数量，常用于生物学分类、社交网络分析和地理数据分析等。

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**

应用场景：适用于具有噪声和不规则簇形状的数据，能够识别具有不同密度的簇。常用于异常检测、图像分割和地理信息系统等。

强化学习

强化学习算法的思路非常简单，以游戏为例，如果在游戏中采取某种策略可以取得较高的得分，那么就进一步**强化**这种策略，以期继续取得较好的结果。这种策略与日常生活中的各种绩效奖励非常类似。我们平时也常常用这样的策略来提高自己的游戏水平。

如何让小鸟飞得更远？

机器有一个明确的小鸟角色——**代理**
需要控制小鸟飞的更远——**目标**
整个游戏过程中需要躲避各种水管——**环境**
躲避水管的方法是让小鸟用力飞一下——**行动**
飞的越远，就会获得越多的积分——**奖励**

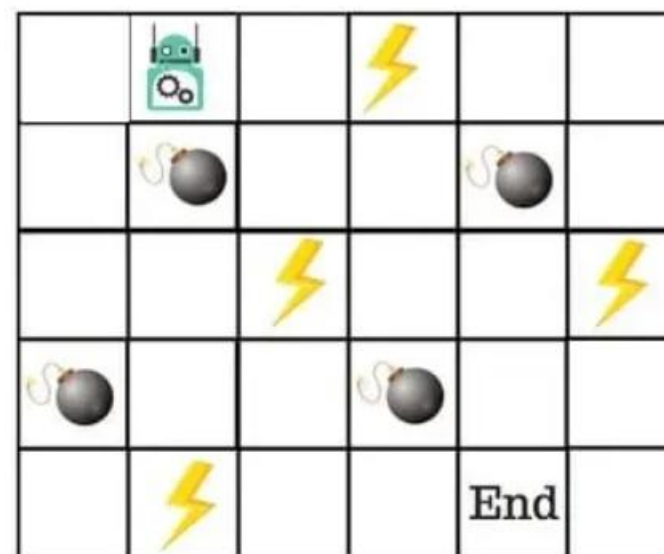
强化学习典型的学习场景



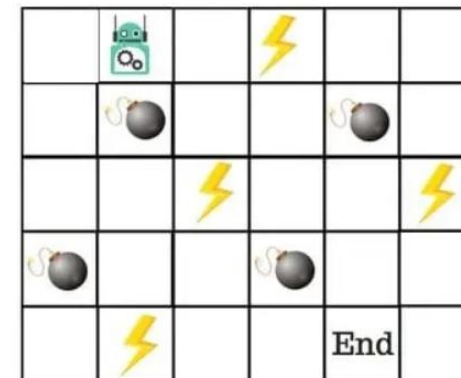
强化学习--Q-learning

Q-Learning是强化学习算法中value-based的算法，Q即为 $Q(s,a)$ 就是在某一时刻的 s 状态下($s \in S$)，采取动作 a ($a \in A$)动作能够获得收益的期望，环境会根据动作反馈相应的回报reward，所以算法的主要思想就是将State与Action构建一张Q-table来存储Q值，然后根据Q值来选取能够获得最大的收益的动作。

任务：训练一个机器人在迷宫中找到宝藏



强化学习--实例



任务：训练一个机器人在迷宫中找到宝藏

环境设置：使用迷宫环境，其中包含了迷宫地图、机器人和宝藏。每个格子可以是墙壁、通道或宝藏，机器人的任务是找到宝藏。

状态 (State)：环境中的每个格子可以看作是一个状态。机器人所处的格子就是它的状态。

动作 (Action)：机器人可以执行的动作有向上、向下、向左、向右移动。

奖励 (Reward)：当机器人移动到宝藏格子时，获得正向奖励；当机器人撞墙或移动到空白格子时，获得负向奖励。目标是通过最大化奖励来找到宝藏。

Q-Table：Q-learning 算法中有一个 Q-Table，用于存储每个状态和动作的 Q 值。Q 值表示在某个状态下执行某个动作的预期奖励。

训练过程：机器人通过与环境进行交互来学习。它在每个状态根据 Q-Table 选择一个动作，并获得奖励和下一个状态。然后使用 Q-learning 算法来更新 Q-Table 中的 Q 值，以优化未来的决策。

提纲

一、机器学习基本概念

二、线性回归

三、支持向量机

四、K-means算法

五、PCA降维

监督学习--线性回归

- 在现实生活中，往往需要分析若干变量之间的关系，如碳排放量与气候变暖之间的关系、某一商品广告投入量与该商品销售量之间的关系等，这种分析不同变量之间存在关系的研究叫回归分析，刻画不同变量之间关系的模型被称为回归模型。如果这个模型是线性的，则称为线性回归模型。
- 一旦确定了回归模型，就可以进行预测等分析工作，如从碳排放量预测气候变化程度、从广告投入量预测商品销售量等。

监督学习--线性回归



监督学习--线性回归

什么是线性回归?

$$y = 33.73(\text{英寸}) + 0.516x$$

y : 子女平均身高

x : 父母平均身高



变量之间符合「线性关系」



建立线性模型，预测结果

- 父母平均身高每增加一个单位, 其成年子女平均身高增加0.516个单位。另外, 他还发现一种“衰退 (regression)”效应 (“回归”到正常人平均身高)。
- 虽然 x 和 y 之间并不总是具有“衰退” (回归) 关系, 但是“线性回归”这一名称就保留下来了。

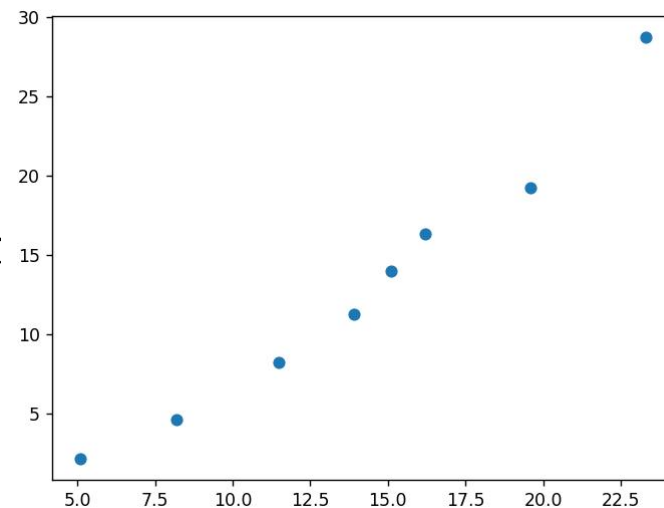
监督学习--线性回归

一元线性回归：

下表给出了芒提兹尼欧（Montesinho）地区发生森林火灾的部分历史数据，表中列举了每次发生森林火灾时的气温温度取值 x 和受到火灾影响的森林面积 y 。

气温温度 x	5.1	8.2	11.5	13.9	15.1	16.2	19.6	23.3
火灾影响面积 y	2.14	4.62	8.24	11.24	13.99	16.33	19.23	28.74

可否对气温温度与火灾所影响的森林面积之间关系进行建模呢？
使用简单的线性模型构建两者之间关系，即气温温度 x 与火灾所影响的森林面积 y 之间的关系，即 $ax + b$ 形式的关系。



监督学习--线性回归

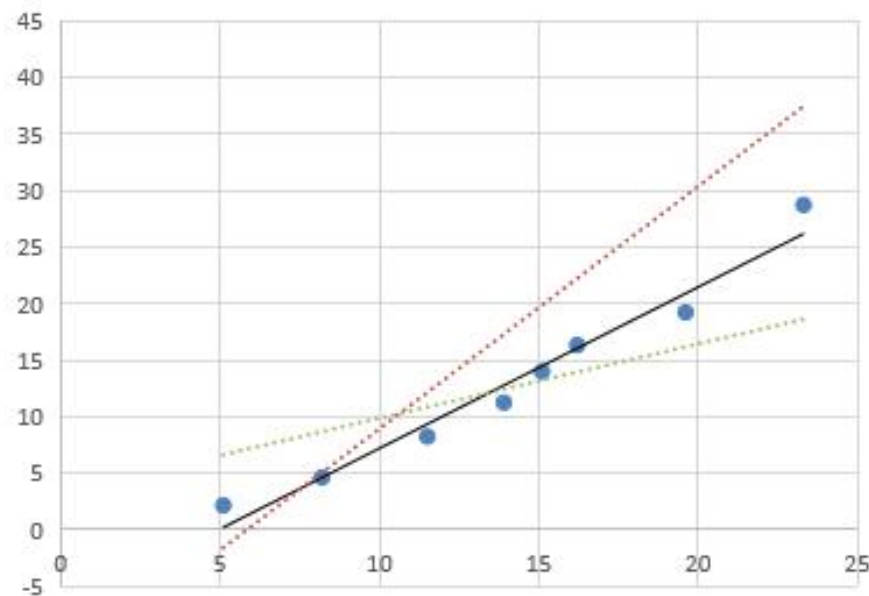
一元线性回归：

气温温度 x	5.1	8.2	11.5	13.9	15.1	16.2	19.6	23.3
火灾影响面积 y	2.14	4.62	8.24	11.24	13.99	16.33	19.23	28.74

线性回归模型： $f(x) = wx + b$

损失函数： $\frac{1}{n} \sum (y - f(x))^2$

如右图，黑实线为最佳线性回归模型



监督学习--线性回归

多元线性回归：

在上述数据中增加一个影响火灾影响面积的潜在因素—风力，该问题就变成了多元线性回归问题。

气温 $x_{i,1}$	5.1	8.2	11.5	13.9	15.1	16.2	19.6	23.3
风力 $x_{i,2}$	4.5	5.8	4	6.3	4	7.2	6.3	8.5
火灾影响面积 y	2.14	4.62	8.24	11.24	13.99	16.33	19.23	28.74

多维数据特征中线性回归的问题定义如下：假设总共有 m 个训练数据 $\{(x_i, y_i)\}_{i=1}^m$ ，其中 $x_i = [x_{i,1}, x_{i,2}, \dots, x_{i,D}] \in \mathbb{R}^D$ ， D 为数据特征的维度，线性回归就是要找到一组参数 $\mathbf{w} = [w_0, w_1, \dots, w_D]$ ，使得线性函数：

$$f(\mathbf{x}_i) = w_0 + \sum_{j=1}^D w_j x_{i,j} = w_0 + \mathbf{w}^T \mathbf{x}_i$$

监督学习--线性回归

多元线性回归：

气温 $x_{i,1}$	5.1	8.2	11.5	13.9	15.1	16.2	19.6	23.3
风力 $x_{i,2}$	4.5	5.8	4	6.3	4	7.2	6.3	8.5
火灾影响面积 y	2.14	4.62	8.24	11.24	13.99	16.33	19.23	28.74

线性回归模型：

$$f(\mathbf{x}_i) = w_0 + \sum_{j=1}^D w_j x_{i,j} = w_0 + \mathbf{w}^T \mathbf{x}_i$$

损失函数：

$$\frac{1}{m} \sum_{i=1}^m (y_i - f(\mathbf{x}_i))^2$$

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# 输入特征和输出目标数据
temperatures = np.array([5.1, 8.2, 11.5, 13.9, 15.1, 16.2, 19.6, 23.3])
winds = np.array([4.5, 5.8, 4.0, 6.3, 4.0, 7.2, 6.3, 8.5])
fire_areas = np.array([2.14, 4.62, 8.24, 11.24, 13.99, 16.33, 19.23, 28.74])
```

监督学习--线性回归

```
# 创建一个三维图形
```

```
fig = plt.figure()
```

```
ax = fig.add_subplot(111, projection='3d')
```

```
# 将输入特征组合成矩阵
```

```
X = np.column_stack((temperatures, winds))
```

```
# 线性回归模型
```

```
model = LinearRegression()
```

```
model.fit(X, fire_areas)
```

```
# 生成平面上的点
```

```
temperatures_mesh, winds_mesh = np.meshgrid(np.linspace(temperatures.min(),  
temperatures.max(), 20), np.linspace(winds.min(), winds.max(), 20))
```

```
X_grid = np.column_stack((temperatures_mesh.ravel(), winds_mesh.ravel()))
```

```
# 预测平面上的点的输出值
```

```
fire_areas_pred = model.predict(X_grid)
```

```
fire_areas_pred = fire_areas_pred.reshape(temperatures_mesh.shape)
```

```
# 绘制散点图
```

```
ax.scatter(temperatures, winds, fire_areas, c='r', marker='o',  
label='Actual Data')
```

```
# 绘制拟合平面
```

```
ax.plot_surface(temperatures_mesh, winds_mesh,  
fire_areas_pred, alpha=0.3, cmap='viridis')
```

```
# 设置坐标轴标签
```

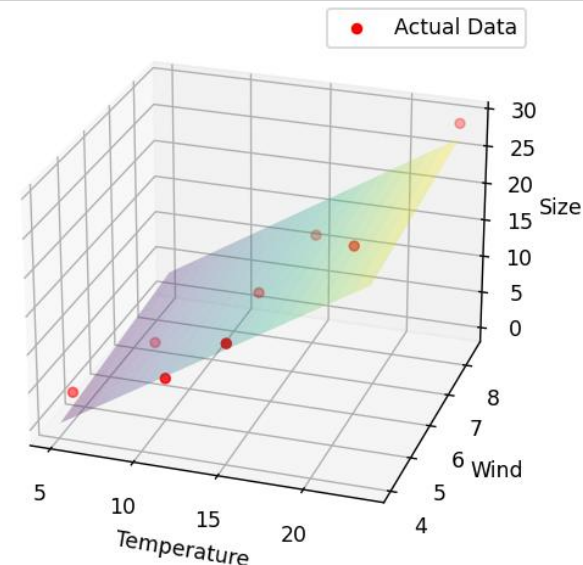
```
ax.set_xlabel('Temperature')
```

```
ax.set_ylabel('Wind')
```

```
ax.set_zlabel('Size')
```

```
plt.legend()
```

```
plt.show()
```



提纲

一、机器学习基本概念

二、线性回归

三、支持向量机

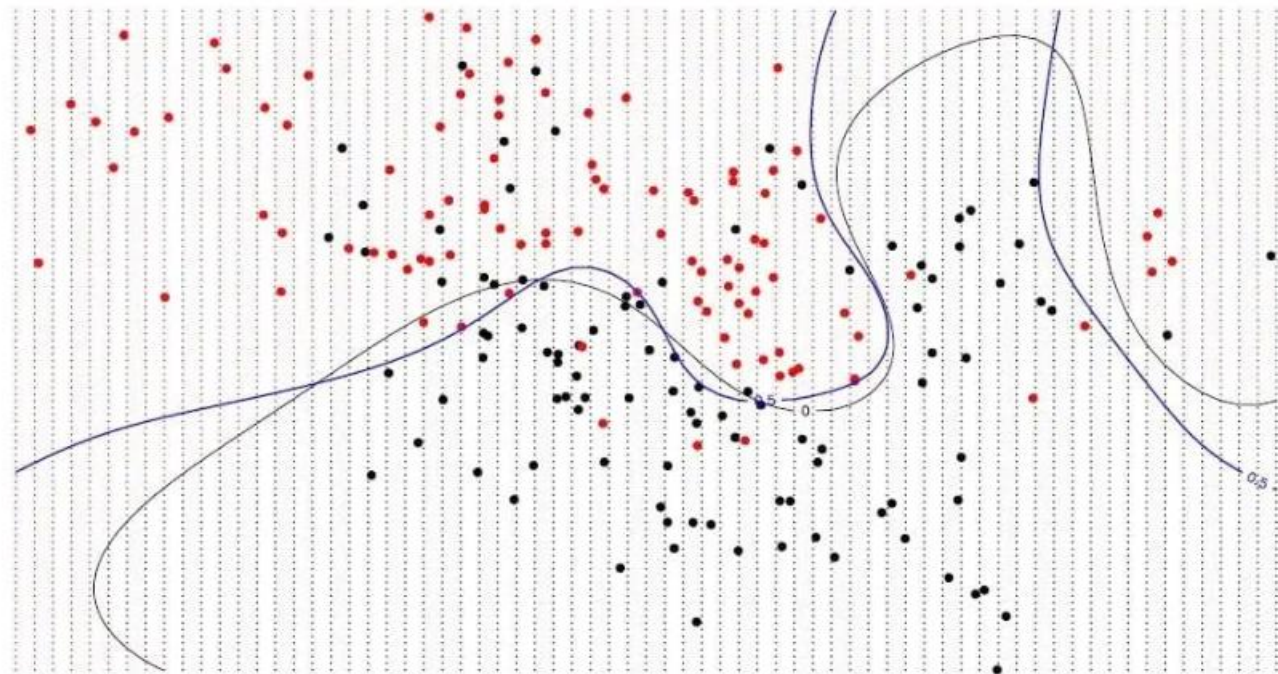
四、K-means算法

五、PCA降维

监督学习--SVM

超平面

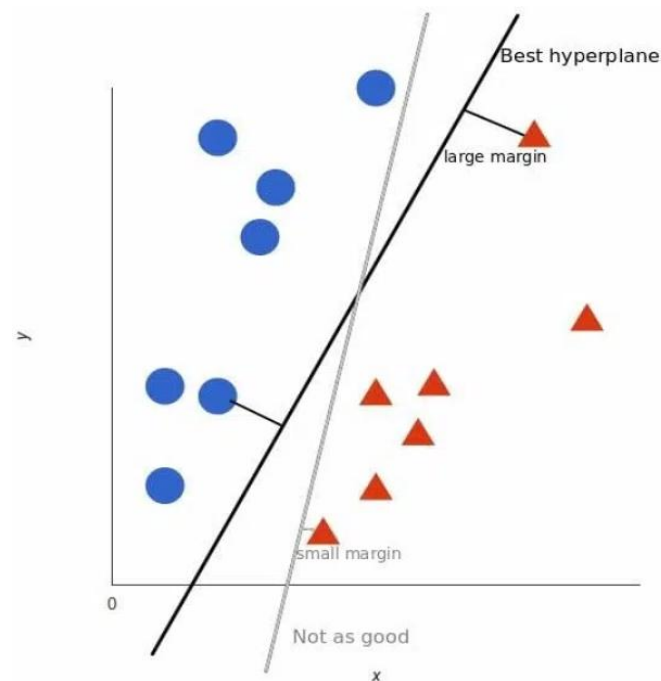
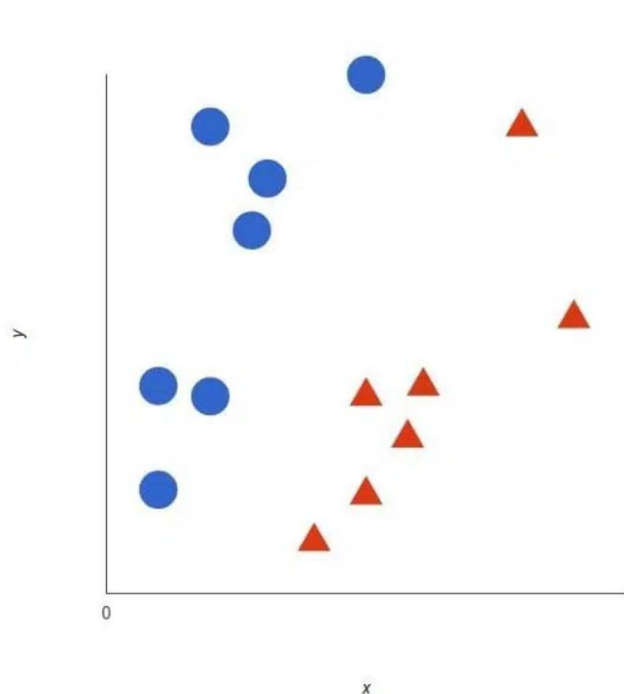
超平面是分割输入变量空间的线。在SVM中，选择超平面以最佳地将输入变量空间中的点与它们的类（0或1）分开。在二维中，您可以将其视为一条线，并假设我们的所有输入点都可以被这条线完全分开。SVM学习算法找到导致超平面最好地分离类的系数。



监督学习--SVM

让我们想象两个类别：红色和蓝色，我们的数据有两个特征：x 和 y。我们想要一个分类器，给定一对 (x, y) 坐标，输出仅限于红色或蓝色。我们将已标记的训练数据列在下图中：

支持向量机会接受这些数据点，并输出一个超平面（在二维的图中，就是一条线）以将两类分割开来。这条线就是判定边界：将红色和蓝色分割开。

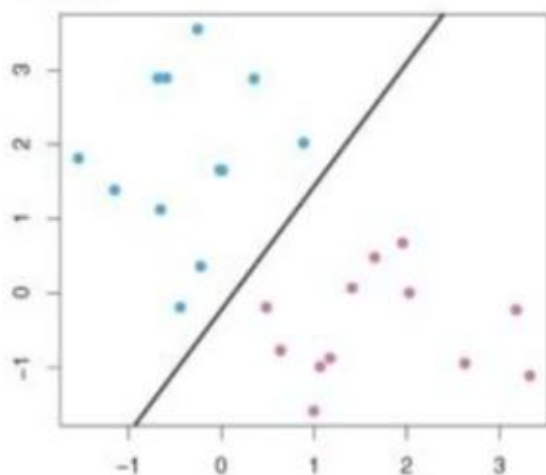


监督学习--SVM

SVM有多种形态，从最简单的线性可分SVM到非线性SVM，再到软间隔非线性SVM，这三种SVM形态是不断进化、不断拓展的过程，能够解决的问题也是从简单逐渐到复杂。

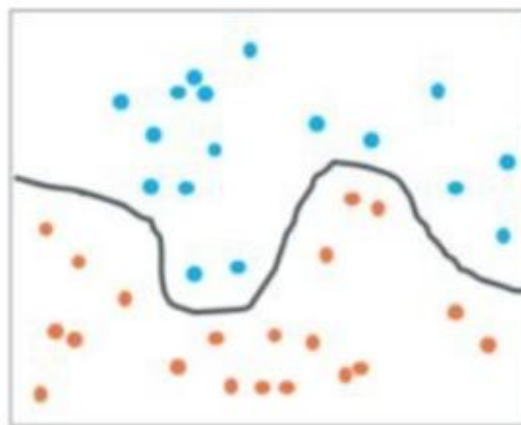
形态1:线性可分SVM

假设数据是完全线性可分的，通过找到一个最优的超平面将数据完美地划分为两个类别



形态2:非线性SVM

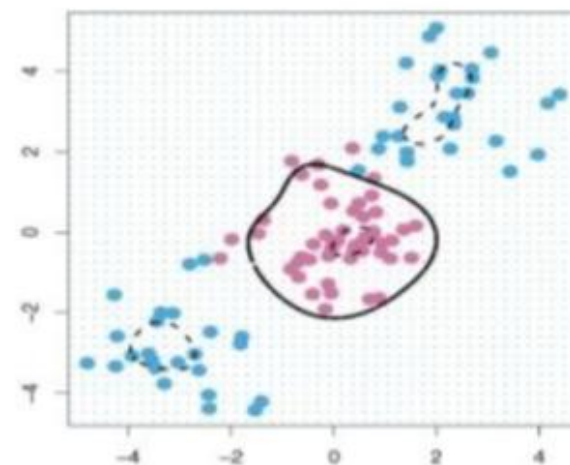
通过引入核函数（如线性核函数，高斯核函数，多项式核函数等），找到非线性的分割面，将数据完美地划分为两个类别。



某种映射函数，将原始特征空间中的数据点转换为一个新的**高维特征空间**，使得原本线性不可分的数据在新的空间中变得线性可分。

形态3:软间隔非线性SVM

通过引入软间隔的概念，不再要求数据完全可分。



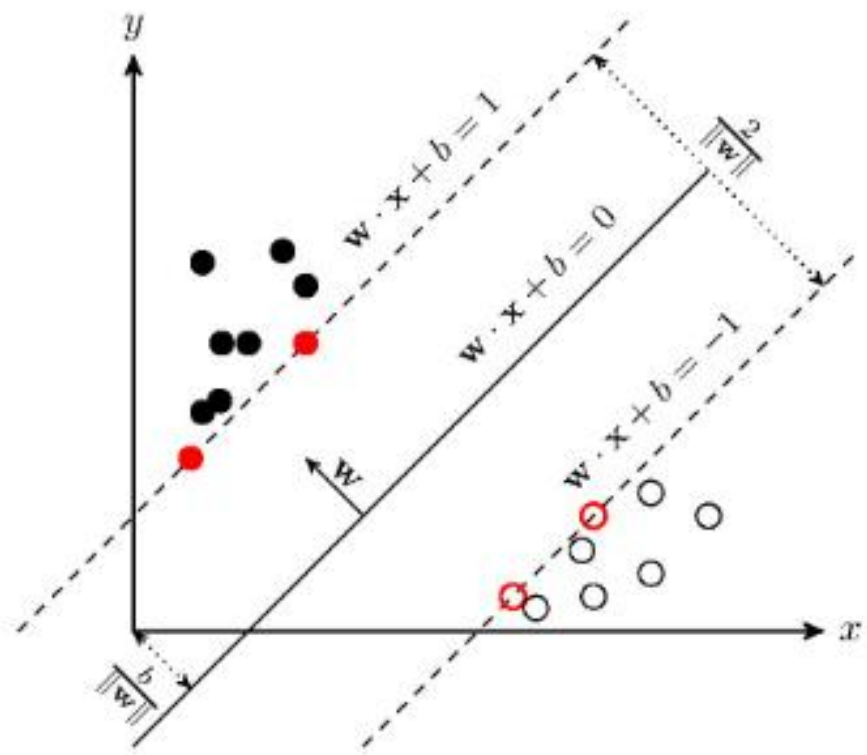
监督学习--SVM

优化目标

参数 w 和 b 确定一个超平面，margin表示样本点离这个超平面的间隔距离，优化目标是从众多的 w 和 b 中找到一个间隔最大的超平面。

需满足的条件(约束)

未引入软间隔的情况下，数据集中每一个样本点需能够被超平面完全分类



监督学习--SVM

线性可分SVM的求解原理

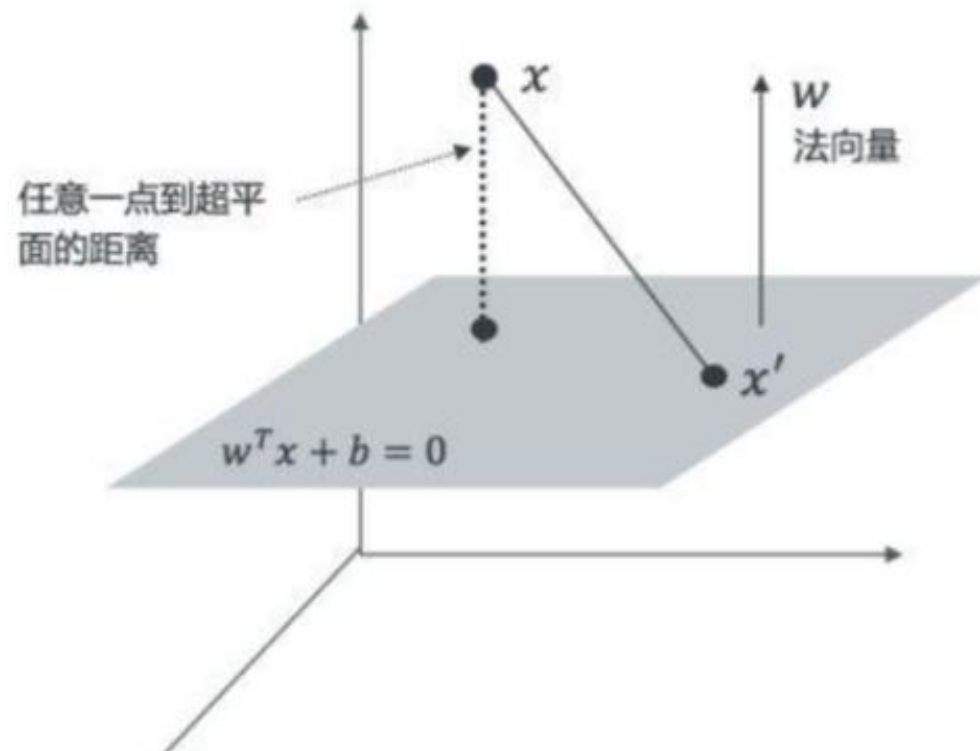
对于给定的数据集 T 和超平面 $w \cdot x + b = 0$ ，定义超平面关于样本点 (x_i, y_i) 的几何间隔为

$$\gamma_i = y_i \left(\frac{w}{\|w\|} \cdot x_i + \frac{b}{\|w\|} \right)$$

其中， $y_i \in \{1, -1\}$ ，代表不同类别。
使点与超平面间隔最大，等价于：

$\max_{w,b} \gamma$ 进一步优化为凸二次规划问题→

$$s.t. \quad y_i \left(\frac{w}{\|w\|} \cdot x_i + \frac{b}{\|w\|} \right) \geq \gamma, i = 1, 2, \dots, N$$



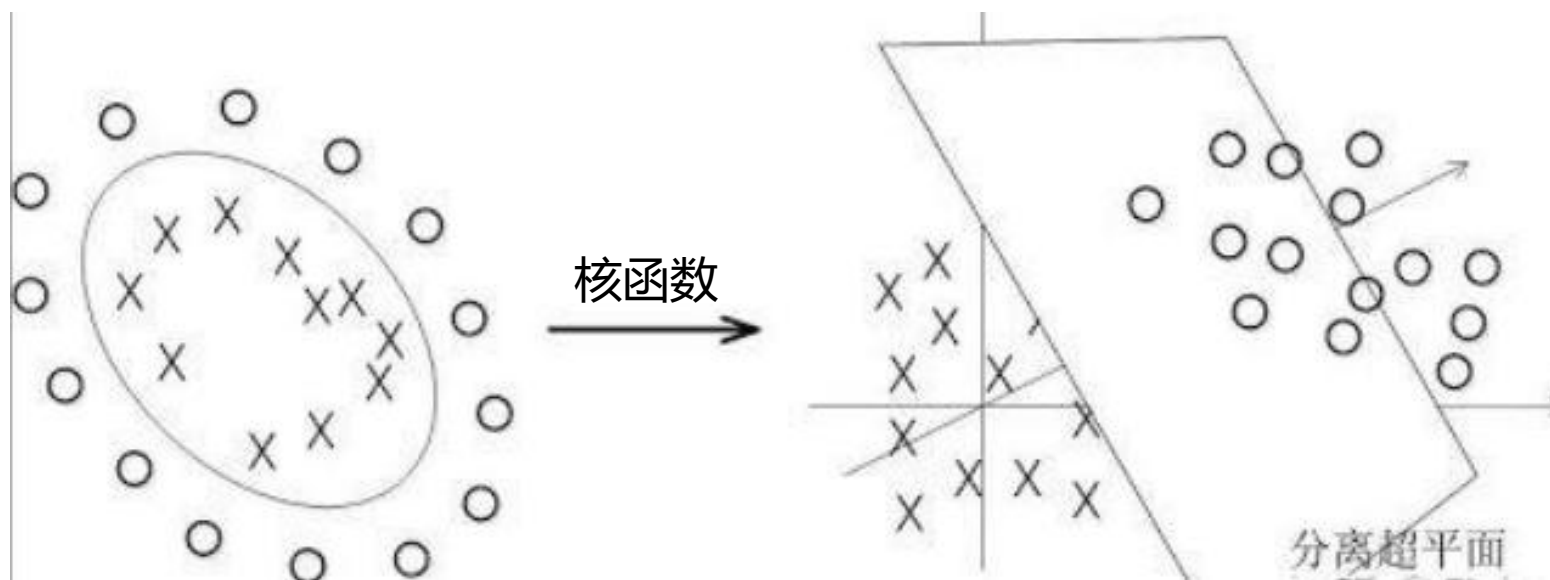
$$\min_{w,b} \frac{1}{2} \|w\|^2$$

$$s.t. \quad y_i (w \cdot x_i + b) \geq 1, i = 1, 2, \dots, N$$

监督学习--SVM

非线性 SVM 分隔超平面

对于在有限维度向量空间中线性不可分的样本，我们使用核函数将其映射到更高维度的向量空间里，再通过间隔最大化的方式，学习得到支持向量机，就是非线性 SVM。

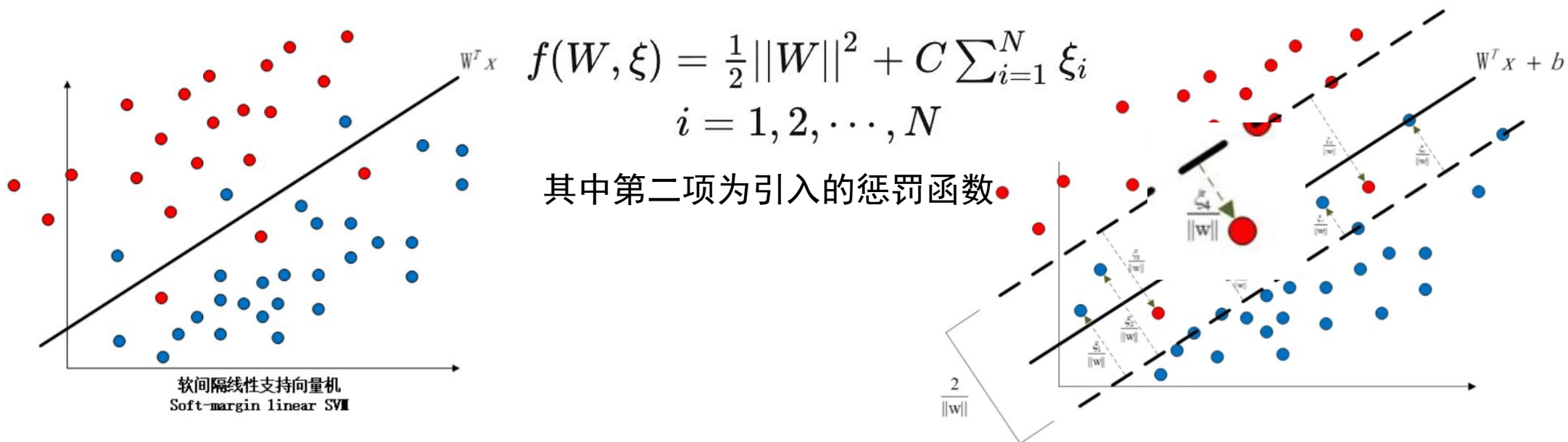


监督学习--SVM

软间隔非线性 SVM

对于这类问题，我们的解决方案很简单，就是在软间隔SVM中，我们的分类超平面既要能够尽可能地将数据类别分对，又要使得支持向量到超平面的间隔尽可能大。

在软间隔SVM中，因为不是完全的线性可分，所以虚线内侧存在有样本点，通过向每一个在虚线内侧的样本点添加松弛变量，将这些样本点搬移到支持向量虚线上。而本身就是在虚线外的样本点的松弛变量则可以设为0，这时的目标函数变为：



监督学习--SVM

在sklearn中使用SVM

```
svm.SVC(  
    C=1.0,  
    kernel='rbf',  
    degree=3,  
    gamma='auto',  
    coef0=0.0,  
    shrinking=True,  
    probability=False,  
    tol=0.001,  
    cache_size=200,  
    class_weight=None,  
    verbose=False,  
    max_iter=-1,  
    decision_function_shape=None,  
    random_state=None  
)
```

关键参数	说明
kernel	核函数
degree	多项式poly函数的维度，默认是3，选择其他核函数时会被忽略
gamma	'rbf'，'poly' 和'sigmoid' 的核函数参数。默认是'auto'
coef0	核函数中的独立项
tol	停止训练的误差值大小，默认为1e-3
cache_size	核函数cache缓存大小，默认为200
max_iter	最大迭代次数。-1为无限制

监督学习--SVM

在sklearn中使用SVM--常用核函数

linear: 线性核函数不进行特征映射，直接在原始特征空间中进行分类。它适用于数据在原始特征空间中就是线性可分的情况，如样本的特征之间有明显的线性关系。

poly: 多项式核函数将数据映射到更高维的特征空间，通过多项式函数来实现。它适用于一些非线性问题，但可能需要调节多项式的阶数，以避免过拟合。

rbf: RBF核函数是应用最广泛的核函数之一。它通过将数据映射到无穷维的特征空间，并基于样本之间的欧氏距离计算内积。RBF核函数适用于非线性、高维数据的分类和回归问题，但需要调节参数来控制映射的复杂度。

sigmoid: Sigmoid核函数将数据映射到一维的特征空间，适用于数据有明显的类别边界但不是线性可分的情况。

Example--SVM

假设我们有一个二分类问题，其中数据点是二维的，分别表示两个特征（特征1和特征2），标签为正类（1）和负类（-1）。我们的目标是使用SVM训练一个分类模型，以便将这些数据点分隔开。

```
# 生成示例数据
```

```
np.random.seed(0)
```

```
X = np.r_[np.random.randn(20, 2) - [2, 2],  
np.random.randn(20, 2) + [2, 2]]
```

```
y = [-1] * 20 + [1] * 20
```

```
# 训练SVM模型
```

```
clf = svm.SVC(kernel='linear')
```

```
clf.fit(X, y)
```

```
# 绘制数据点和分隔超平面
```

```
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired)
```

```
ax = plt.gca()
```

```
xlim = ax.get_xlim()
```

```
ylim = ax.get_ylim()
```

```
# 生成网格来绘制决策边界
```

```
xx, yy = np.meshgrid(np.linspace(xlim[0], xlim[1], 50),  
np.linspace(ylim[0], ylim[1], 50))
```

```
Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
```

```
Z = Z.reshape(xx.shape)
```

```
# 绘制分隔超平面和间隔边界
```

```
ax.contour(xx, yy, Z, colors='k', levels=[-1, 0, 1], alpha=0.5,  
linestyles=['--', '-', '--'])
```

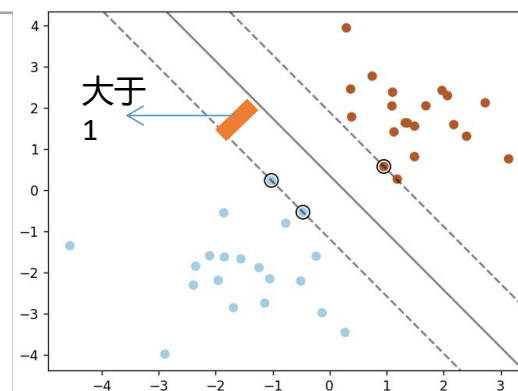
```
# 绘制支持向量
```

```
ax.scatter(clf.support_vectors_[:, 0],
```

```
clf.support_vectors_[:, 1], s=100,
```

```
linewidth=1, facecolors='none', edgecolors='k')
```

```
plt.show()
```



提纲

一、机器学习基本概念

二、线性回归

三、支持向量机

四、K-means算法

五、PCA降维

无监督学习--K-means

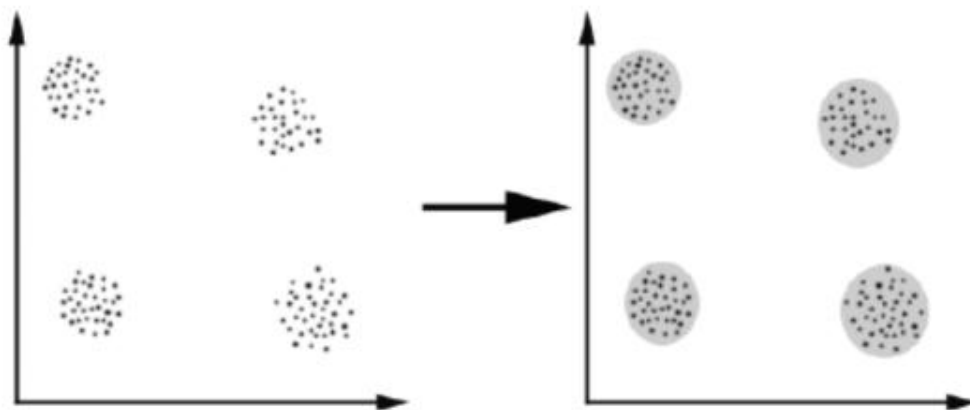
K-means聚类算法是**基于距离**的聚类算法(cluster algorithm)

K-means的思想

- 将样本集合划分为k个子集，构成k个类
- 将n个样本分到k个类中，每个样本到其所属类的中心距离最小

K-means的主要步骤

- 第一步：初始化聚类中心
- 第二步：给聚类中心分配样本
- 第三步：移动聚类中心
- 第四步：重复二、三步



无监督学习--K-means

第一步:初始化聚类中心

➤ k 需要提前确定

- 随机选取k个样本作为初始的聚类中心

第二步:给聚类中心分配样本

➤ 初始中心的选择影响聚类结果

- 计算每个样本与各个聚类中心之间的距离，把每个样本分配给距离它最近的聚类中心。

第三步:移动聚类中心

➤ 可能存在空簇

- 新的聚类中心移动到这个聚类所有样本的平均值处。

第四步:重复2, 3步直到结果稳定

无监督学习--K-means

```
| 获取数据 n 个 m 维的数据  
| 随机生成 k 个 m 维的点  
| while(t)  
|   for(int i=0;i < n;i++)  
|     for(int j=0;j < k;j++)  
|       计算点 i 到类 j 的距离  
|   for(int i=0;i < k;i++)  
|     1. 找出所有属于自己这一类的所有数据点  
|     2. 把自己的坐标修改为这些数据点的中心点坐标  
| end
```

复杂度

$O(tknm)$: 其中, t 为迭代次数, k 为簇的数目, n 为样本点数, m 为样本点维度。

无监督学习--K-means

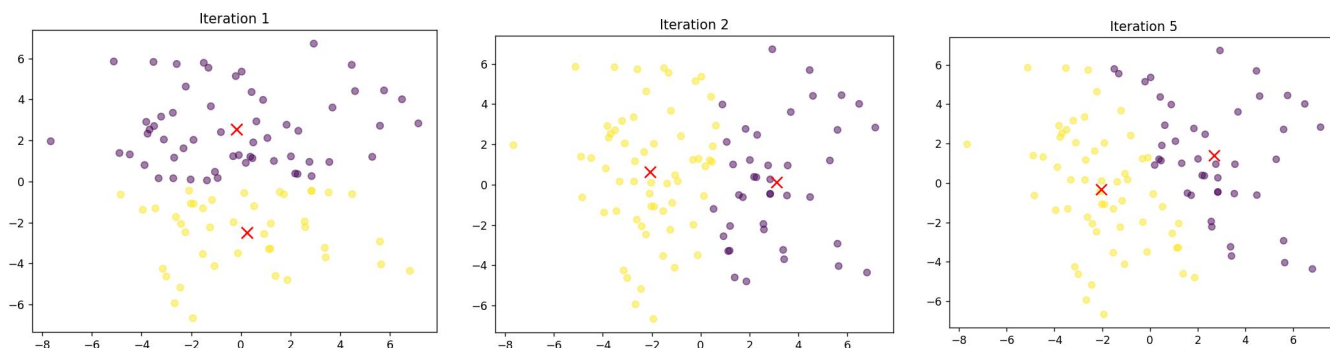
下面使用一个简单的例子说明K-means聚类算法的流程
假设我们有100个随机生成的二维数据点，选择聚类中心数 $K=2$

初始化：选择初始簇中心。

分配数据点：计算每个点与两个簇中心的距离，将其分配到更近的簇中。

更新簇中心：计算每个簇内数据点的平均值，更新簇中心。

重复步骤2和3：重复执行步骤2和3，直到簇中心稳定。



```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

创建示例数据点

```
np.random.seed(0)
data = np.random.randn(100, 2)
```

初始化 KMeans 模型

```
kmeans = KMeans(n_clusters=2, init='random', n_init=1)
```

迭代绘制结果

```
for i in range(5): # 假设迭代 5 次
    kmeans.fit(data)
```

绘制数据点和当前聚类中心

```
plt.scatter(data[:, 0], data[:, 1], c=kmeans.labels_,
            cmap='viridis', alpha=0.5)
plt.scatter(kmeans.cluster_centers_[0, 0],
            kmeans.cluster_centers_[0, 1], marker='x', c='red', s=100)
plt.title(f"Iteration {i + 1}")
plt.show()
```

无监督学习--K-means

了解了大致流程后，动手试试吧~

```
def k_means(X, k, max_iters=100):
    # 获取样本总数与维度
    n_samples, n_features = X.shape
    # 随机选择k个样本作为初始质心
    centroids = X[np.random.choice(n_samples, k, replace=False)]
    labels = np.zeros(n_samples)
    for _ in range(max_iters):
        # 计算每个样本到每个质心的欧氏距离，为样本分配新的簇标签
        distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
        new_labels = np.argmin(distances, axis=1)
        # 判断算法是否收敛，若结果不变，终止运行
        if np.array_equal(new_labels, labels):
            break
        labels = new_labels
        for i in range(k):
            centroids[i] = np.mean(X[labels == i], axis=0)
    return labels, centroids
```

现有200个点的数据集，请你用K-means算法将其分类

生成测试数据

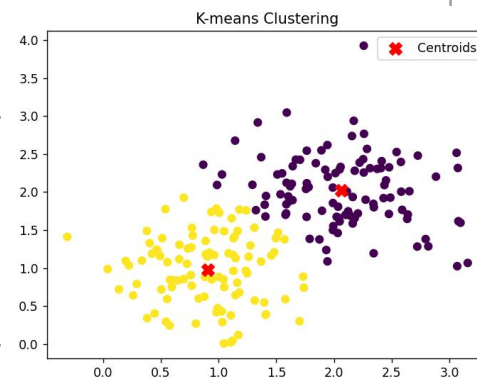
```
np.random.seed(42)
circle1 = np.random.normal(loc=[1, 1], scale=0.5,
                             size=(100, 2))
circle2 = np.random.normal(loc=[2, 2], scale=0.5,
                             size=(100, 2))
X = np.vstack((circle1, circle2))
```

执行K-means算法

```
k = 2
labels, centroids = k_means(X, k)
```

绘制散点图

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], c='red',
            marker='X', s=100, label='Centroids')
plt.title('K-means Clustering')
```



提纲

一、机器学习基本概念

二、线性回归

三、支持向量机

四、K-means算法

五、PCA降维

PCA降维

- 将高维度的数据保留下最重要的一些特征
- 去除噪声和不重要的特征
- 实现提升数据处理速度的目的

主成分分析

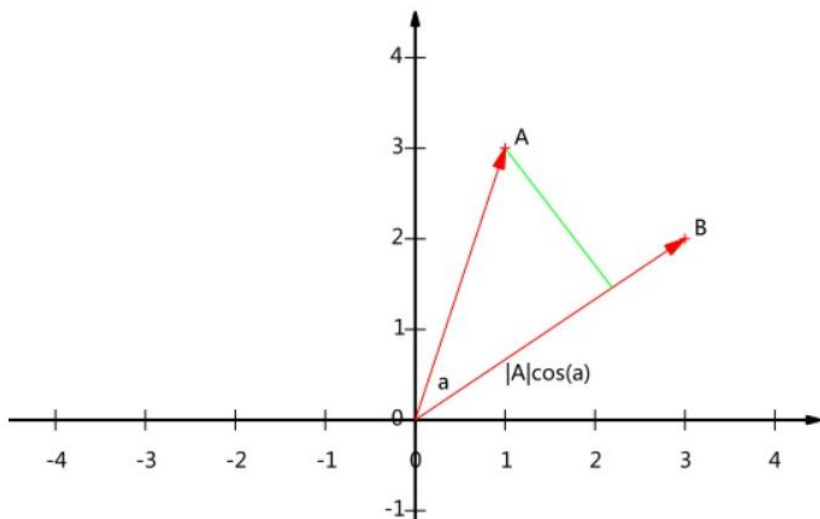
- 一种最常用的无监督降维方法，通过降维把多个变量化为少数几个主成分的统计分析方法
- 这些主成分能够反映原始变量的绝大部分信息，它们通常表示为原始变量的某种线性组合

PCA降维

主成份分析 (Principal Component Analysis, PCA) 是一种利用线性映射来进行数据降维的方法，并去除数据的相关性；且最大限度保持原始数据的方差信息。

要准确描述向量，首先要确定一组基，然后给出在基所在的各个直线上的投影值。如果基的数量少于向量本身的维数，则可以达到降维的效果。

如果我们有一组 N 维向量，现在要将其降到 K 维 (K 小于 N)，我们希望投影后的投影值尽可能分散，因为如果重叠就会有样本消失。



基变换的矩阵表示：

$$\begin{pmatrix} p_1 \\ p_2 \\ \vdots \\ p_R \end{pmatrix} (a_1 \quad a_2 \quad \cdots \quad a_M) = \begin{pmatrix} p_1 a_1 & p_1 a_2 & \cdots & p_1 a_M \\ p_2 a_1 & p_2 a_2 & \cdots & p_2 a_M \\ \vdots & \vdots & \ddots & \vdots \\ p_R a_1 & p_R a_2 & \cdots & p_R a_M \end{pmatrix}$$

PCA降维

在一维空间中我们可以用方差来表示数据的分散程度。而对于高维数据，我们用协方差进行约束，协方差可以表示两个变量的相关性。为了让两个变量尽可能表示更多的原始信息，我们希望它们之间不存在线性相关性，因为相关性意味着两个变量不是完全独立，必然存在重复表示的信息。

因此，我们使用PCA降维的目标可以抽象为：

各变量两两间协方差为 0，而每个变量方差则尽可能大

假设我们只有 a 和 b 两个变量，那么我们将它们按行组成矩阵 X：
为了方便处理，我们将每个变量的均值都化为 0

$$X = \begin{pmatrix} a_1 & a_2 & \cdots & a_m \\ b_1 & b_2 & \cdots & b_m \end{pmatrix}$$

可以看出下方矩阵包含了变量方差与变量间协方差的信息，
我们要做的是确保其为对角阵，且对角线上的元素从大到小排序

$$\frac{1}{m} X X^T = \begin{pmatrix} \frac{1}{m} \sum_{i=1}^m a_i^2 & \frac{1}{m} \sum_{i=1}^m a_i b_i \\ \frac{1}{m} \sum_{i=1}^m a_i b_i & \frac{1}{m} \sum_{i=1}^m b_i^2 \end{pmatrix} = \begin{pmatrix} Cov(a, a) & Cov(a, b) \\ Cov(b, a) & Cov(b, b) \end{pmatrix}$$

PCA降维

设原始数据矩阵 X 对应的协方差矩阵为 C ，而 P 是一组基按行组成的矩阵，设 $Y = PX$ ，则 Y 为 X 对 P 做基变换后的数据。设 Y 的协方差矩阵为 D ，我们推导一下 D 与 C 的关系

$$\begin{aligned} D &= \frac{1}{m} Y Y^T \\ &= \frac{1}{m} (PX)(PX)^T \\ &= \frac{1}{m} P X X^T P^T \\ &= P \left(\frac{1}{m} X X^T \right) P^T \\ &= P C P^T \end{aligned}$$

因此，要使 D 为对角阵，
我们使用PCA降维的目标可以进一步简化为：

- 求出协方差矩阵 C 的特征值及对应的特征向量；
- 将特征向量按对应特征值大小从上到下按行排列成矩阵；
- 取前 k 个特征向量组成矩阵 P ， P 则为线性变换的基。

PCA降维--步骤

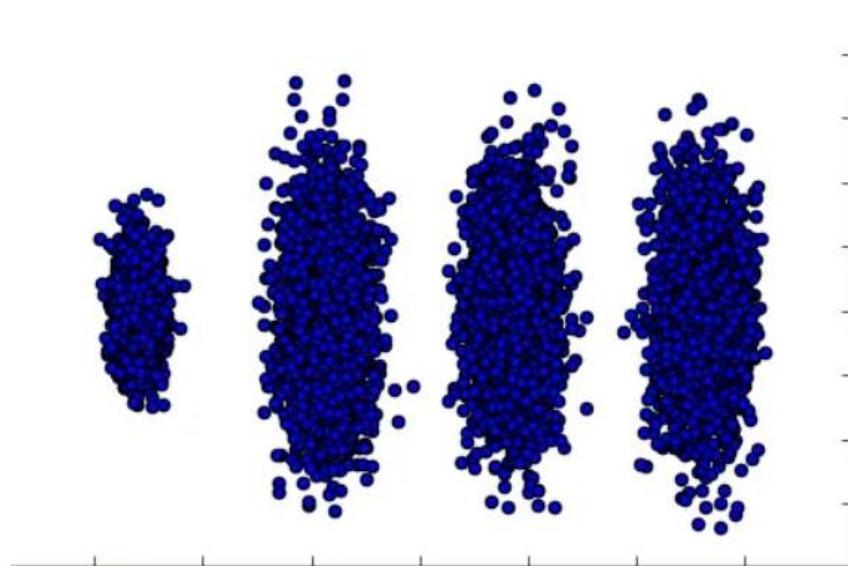
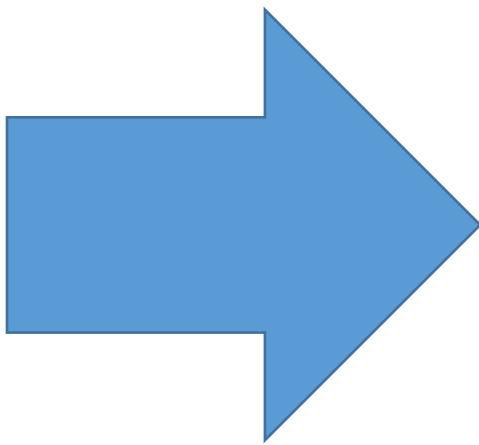
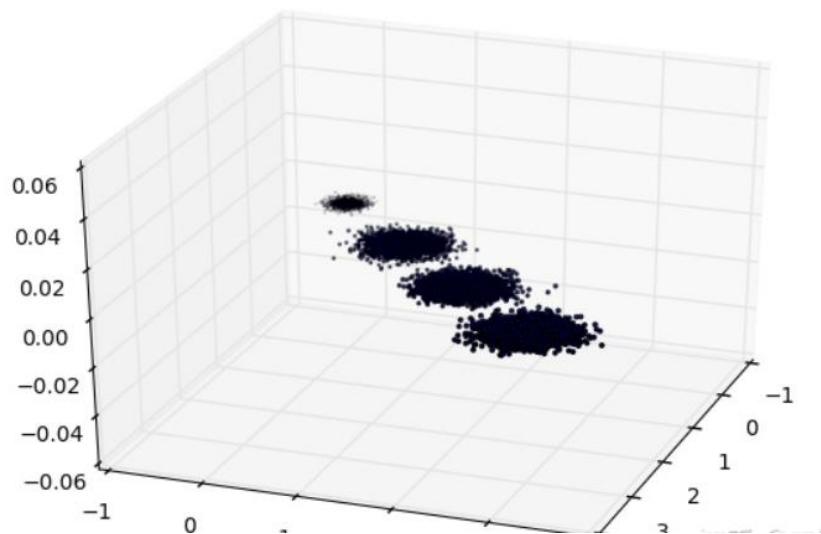
设有 m 条 n 维数据

- 将原始数据按列组成 n 行 m 列矩阵 X
- 将 X 的每一行进行零均值化，即减去这一行的均值
- 求出协方差矩阵 $C = \frac{1}{m}XX^T$
- 求出协方差矩阵的特征值及对应的特征向量
- 将特征向量按对应特征值大小从上到下按行排列成矩阵，取前 k 行组成矩阵 P
- $Y = PX$ 即为降维到 k 维后的数据

PCA降维--实例

将三维数据降为二维并保留其超过95%的特征

此处近似为选取 $(\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2}, 0)$ 的基，即为在x-y平面的投影较大地保留了原始数据的特征



PCA降维--实例

现有一个分类工作，需要根据一组二维实体描述数据对数据进行分类，由于处理器限制，需要将二维数据转换为一维再进行分类，以提高效率。

生成二维数据

```
np.random.seed(42)
mean = [0, 0]
cov = [[1, 0.8], [0.8, 1]]
X = np.random.multivariate_normal(mean, cov, 100)
```

绘制原始数据散点图

```
plt.scatter(X[:, 0], X[:, 1], alpha=0.6)
plt.title('Original Data')
plt.show()
```

使用PCA降维到一维

```
pca = PCA(n_components=1)
X_pca = pca.fit_transform(X)
```

将降维后的数据恢复到原始维度

```
X_reconstructed = pca.inverse_transform(X_pca)
```

绘制降维后数据恢复的效果

```
plt.scatter(X_reconstructed[:, 0], X_reconstructed[:, 1],
            alpha=0.6)
plt.title('Processed Data')
plt.show()
```

